Prior Art Search and Patentability Analysis for the "Forgetting Engine" Algorithm

Executive Summary: Patentability Assessment of the "Forgetting Engine"

This report presents a detailed analysis of the prior art relevant to the "Forgetting Engine" algorithm, a novel population-based optimization method. The search has identified several key areas of art, most notably **Extremal Optimization (EO)** and **Viability Evolution**, which exhibit conceptual and mechanistic overlaps with the invention. However, a thorough review of academic literature, patent databases, and other sources indicates that no single reference or combination of references fully anticipates or renders obvious the complete inventive concept as claimed.

The novelty of the Forgetting Engine resides not in the individual concepts of "elimination" or "diversity retention" in isolation, but in their specific, synergistic synthesis. The core patentable feature appears to be the combination of: (a) aggressive, percentage-based culling of the *worst-performing candidate solutions* from a population as the primary iterative step, with (b) an explicit, feature-aware "paradox retention" filter that rescues otherwise-doomed candidates based on their structural promise or novelty. This combination creates a unique dynamic that balances exploitation (via aggressive culling) and exploration (via paradox retention), leading to demonstrably superior performance on complex optimization problems.

The most significant prior art references that must be carefully considered and distinguished during patent prosecution are **Extremal Optimization (EO)** by Boettcher and Percus and its derivatives, and the **"Artificial Evolution by Viability Rather than Competition"** algorithm by Maesani et al. (2014). While these references share a philosophical leaning towards elimination-based dynamics, they differ fundamentally in their mechanism, level of application, and ultimate objective. EO operates on the components of a single solution, not a population of solutions, and Viability Evolution uses hard constraints for elimination rather than ranked performance and lacks the critical paradox retention feature.

Based on this analysis, the preliminary opinion is that the Forgetting Engine possesses strong arguments for both novelty and non-obviousness. The invention's unique synthesis of aggressive, rank-based culling with a specific, feature-aware paradox retention mechanism addresses a known problem in evolutionary computation (the trade-off between convergence speed and diversity) in a manner not taught or suggested by the prior art.

The Inventive Concept: Defining the "Forgetting Engine"

Precise Characterization of the Invention

Based on the provided internal documentation, including a provisional patent application and experimental reports, the Forgetting Engine is a population-based metaheuristic algorithm designed for solving complex, NP-hard optimization problems. Its philosophical underpinning is described as "creation through negation," a paradigm where the optimal solution is revealed by strategically and aggressively eliminating non-viable pathways rather than by searching for viable ones. This approach is framed as making "forgetting an active computational primitive," which distinguishes it from traditional search-based methods like Monte Carlo simulation or selection-based methods like Genetic Algorithms.
The algorithm's architecture is designed to maintain a dynamic balance between intense selective pressure (exploitation) and the preservation of exploratory potential (exploration). This is achieved through a novel iterative loop that combines two opposing but synergistic mechanisms: the aggressive culling of poor-performing solutions and the deliberate retention of certain "paradoxical" solutions that, while performing poorly, exhibit features suggesting long-term promise.

Distillation of Key Claim Elements

The core, patentable features of the algorithm are explicitly defined in the provisional patent application and validated by the experimental data. These elements form the basis of the invention's unique contribution to the field of computational optimization.

1. **Forgetting-First / Elimination as Primitive:** The primary and most aggressive computational step in each iteration is the permanent elimination of a predefined percentage of the population. Specifically, the algorithm evaluates all candidate solutions in the population, ranks them according to a fitness or energy metric, and permanently discards the worst-performing R%. The preferred embodiment, validated in experiments, specifies a forget_rate of 30%. This "delete the worst" approach is the engine's main driver of convergence and is fundamentally different from methods that select the *best* individuals for survival or reproduction.
2. **Paradox Retention Mechanism:** This is a critical and distinguishing feature. A specific filter is applied *prior to the elimination step* to identify and protect certain candidates that would otherwise be culled. These are candidates that fall within the worst-performing R% but exhibit contradictory or paradoxical properties suggesting they are on a promising, albeit difficult, path to an optimal solution. The mechanism is not random but is based on specific, measurable criteria :
    - **Structural Contradiction Score (SCS):** Measures the structural similarity of a candidate to known optimal solution features, protecting it even if its current energy/fitness is poor.
    - **Energetic Paradox Indicator (EPI):** Identifies candidates with locally high energy but situated in regions of the solution space known to contain global optima.
    - **Diversity Contribution Metric (DCM):** Quantifies a candidate's contribution to the overall structural or genetic diversity of the population, protecting it to prevent premature convergence. This explicit, feature-aware retention of low-fitness but high-potential outliers is a key innovation designed to escape local optima and solve deceptive problems.
3. **Population-Based Parallel Exploration:** The algorithm operates on a population of N candidate solutions, with a preferred embodiment of N=50. This parallel exploration of the solution space is a core architectural choice. After the elimination and retention steps, the population is replenished back to its original size N primarily through the mutation of

surviving candidates, ensuring the continuation of promising genetic material. This architecture distinguishes it from single-solution search methods like Simulated Annealing or canonical Extremal Optimization.

4. **Application Domain and Performance:** The invention has been rigorously validated on the 2D HP (Hydrophobic-Polar) lattice protein folding problem, a well-established benchmark for NP-hard combinatorial optimization. The experimental results demonstrate statistically significant outperformance against a standard Monte Carlo baseline. The Forgetting Engine achieved an 80% higher success rate (45% vs. 25%), converged 2.15 times faster (average of 367 steps vs. 789), and found superior final energy states, with all improvements having a p-value of less than 0.001.

Prior Art Landscape: A Categorical Analysis

The prior art landscape relevant to the Forgetting Engine is fragmented, with different fields of computer science and computational physics developing algorithms that touch upon individual components of the invention but not its complete, integrated system. Concepts like elimination, pruning, and diversity maintenance are common in the art. However, the specific synthesis of these concepts into the Forgetting Engine's core loop appears to be unique. To systematically map this landscape and identify the novelty of the invention, a comprehensive comparison is necessary. The following table provides a high-level overview of the major classes of prior art and their relevance to the key features of the Forgetting Engine.

Reference / Class	Core Algorithm Type	Forgetting-First Primitive (Eliminates worst R% of population?)	Paradox Retention (Explicitly retains low-fitness, high-novelty candidates?)	Population-Based?	Key Application(s)	Relevance Score
Forgetting Engine	**Inventive Concept**	**Yes** (Primary step is eliminating worst R% of solutions)	**Yes** (Explicit, feature-based retention filter)	**Yes**	Combinatorial Optimization (Protein Folding)	**N/A**
Extremal Optimization (EO)	Single-Solution Heuristic	**Partial** (Eliminates worst *component* of *one* solution)	**No** (Exploration via "avalanches", not explicit retention)	**No**	Combinatorial Optimization (Spin Glasses)	**High**
Viability Evolution	Population-Based EA	**Yes** (Eliminates fraction of population violating constraints)	**No** (No mechanism to rescue non-viable candidates)	**Yes**	Design Optimization	**High**
Quality-Diversity (QD) /	Illumination Algorithm	**No** (Fills an archive; does	**No** (Diversity via	**Yes** (Population	Robotics, Procedural	**Medium**

Reference / Class	Core Algorithm Type	Forgetting-First Primitive (Eliminates worst R% of population?)	Paradox Retention (Explicitly retains low-fitness, high-novelty candidates?)	Population-Based?	Key Application(s)	Relevance Score
MAP-Elites		not cull a population)	behavioral niches, not paradox retention)	is the archive)	Content Generation	
Truncation Selection	EA Selection Operator	**Yes** (Selects top (100-R)%, equivalent to culling worst R%)	**No** (Known to destroy diversity; lacks retention mechanism)	**Yes**	General Evolutionary Algorithms	**Medium**
Lexicase Selection	EA Parent Selection	**No** (Selects parents for reproduction, not survivors)	**Partial** (Implicitly retains "specialists")	**Yes**	Genetic Programming	**Low**
Beam Search	Heuristic Search	**No** (Keeps absolute top-K paths, not relative worst R%)	**No** (Some variants add diversity, but no paradox retention)	**No** (Operates on a "beam" of paths)	NLP, Pathfinding	**Low**
Negative Selection Algorithm (NSA)	Anomaly Detection	**No** (Terminological overlap only; eliminates bad "detectors")	**No**	**No** (Operates on a set of detectors, not solutions)	Cybersecurity, Fault Detection	**Very Low**

Extremal Optimization (EO)

The foundational Extremal Optimization (EO) algorithm, developed by Boettcher and Percus around 2000, is a significant piece of prior art due to its shared philosophical motivation with the Forgetting Engine. Inspired by the Bak-Sneppen model of self-organized criticality in evolution, EO operates on the principle of selecting *against the bad* rather than *for the good*. The core mechanism of canonical EO involves analyzing a *single candidate solution* and assigning a "fitness" value to each of its individual components or variables. In each iteration, the algorithm identifies the single *worst component* within that solution and forces it to change its state, typically to a new random value. This change is accepted unconditionally, which can lead to large, "avalanche-like" fluctuations in the overall solution quality, enabling the search to escape local optima. EO has been successfully applied to hard combinatorial problems, including spin glasses and protein folding.

The primary distinction from the Forgetting Engine is the level of abstraction at which the

"elimination" principle is applied. Canonical EO is a single-solution heuristic that modifies *parts of one solution*. The Forgetting Engine is a population-based algorithm that eliminates *entire solutions from a population*. This represents a fundamental architectural difference.
Furthermore, EO lacks an explicit "paradox retention" mechanism. While its stochastic nature allows for broad exploration, it does not contain a specific, deterministic rule to identify and protect globally promising candidates based on structural or other features.
The existence of "Population-Based EO" (PEO) variants must also be considered. These represent attempts to combine the power of EO's local search with the global exploration benefits of a population. However, a detailed analysis of these methods, such as the Adaptive Co-evolution Population-based Extremal Optimization (ACPEO) algorithm, reveals a different approach. These algorithms typically apply an EO-like local search operator to *each individual* within the population, often in a hybrid fashion with other evolutionary operators like Differential Evolution. They do not implement the Forgetting Engine's core population-wide dynamic of evaluate all -> identify worst R% -> eliminate them -> replenish. The development of PEO in the literature shows that when faced with the challenge of scaling EO to a population, the path taken by researchers was to use EO as a sophisticated mutation operator within a traditional evolutionary framework. This path is distinct from the Forgetting Engine's approach, which elevates the *philosophy* of EO to become the primary population management operator itself. This distinction is a strong basis for an argument of non-obviousness.

Viability and Culling-Based Evolutionary Algorithms

The 2014 paper "Artificial Evolution by Viability Rather than Competition" by Maesani et al. is a highly relevant reference. It introduces a "Viability Evolution" (ViE) algorithm designed to maintain high population diversity and find a large number of unique solutions that satisfy a set of minimal requirements. The core mechanism of ViE involves defining a set of "viability boundaries" for the problem's objectives. At discrete intervals, these boundaries are dynamically tightened such that a user-defined fraction of the population (e.g., 25% in their experiments) is rendered non-viable and is eliminated. This mechanism presents a strong conceptual overlap with the Forgetting Engine's "Forgetting-First" primitive.
Despite this similarity, there are critical distinctions. The first lies in the criterion for elimination. ViE eliminates individuals based on a binary, all-or-nothing satisfaction of hard constraints (the viability boundaries). An individual is either inside the boundaries and viable, or outside and eliminated. All individuals that remain within the boundaries are considered equally viable and are selected for reproduction uniformly. In contrast, the Forgetting Engine ranks the entire population based on a continuous performance metric (like energy or fitness) and eliminates the *relative worst-performing* R%. This rank-based culling is a more nuanced form of selection pressure.
The most crucial difference, however, is the complete absence of a paradox retention mechanism in ViE. The ViE algorithm has no described method for rescuing a solution that falls outside the viability boundary, regardless of its structural novelty or other promising features. This absence is not merely a missing feature but points to a fundamental difference in the algorithms' objectives. The stated goal of ViE is to find "the largest number of different solutions satisfying minimal requirements"—it is an *illumination* or *exploration* algorithm designed to map out the space of viable solutions. The Forgetting Engine, as demonstrated on the protein folding problem, is an *optimization* algorithm designed to find a single optimal or near-optimal solution. For this optimization task, especially in deceptive energy landscapes, an algorithm must be able to tolerate temporarily "bad" solutions that are on a path to a globally "great" solution. The

paradox retention mechanism is precisely the component that enables this tolerance. Therefore, the combination of culling *and* paradox retention is a unique synthesis tailored for a different and arguably harder class of problem than that addressed by ViE.

Quality-Diversity (QD) and Illumination Algorithms

Quality-Diversity (QD) algorithms represent a significant branch of evolutionary computation that explicitly prioritizes the discovery of a wide range of high-performing solutions. The goal of a QD algorithm, such as the well-known MAP-Elites (Multi-dimensional Archive of Phenotypic Elites), is to "fill a space of possibilities with the best possible example of each type of achievable behavior". The core mechanism involves defining a "behavior space" (e.g., for a robot, the dimensions could be speed and leg movement frequency) and discretizing this space into a grid or archive. As new solutions are generated, they are mapped to a specific cell in this grid based on their behavior. If that cell is empty, or if the new solution has a higher fitness ("quality") than the current occupant, it replaces the occupant. The final output of the algorithm is not a single solution but the entire archive, which contains a diverse collection of "elites," one for each behavioral niche.

The operational mechanism of QD is fundamentally different from that of the Forgetting Engine. QD is an "archive-and-fill" or "store-and-replace" process, not a "cull-and-replenish" process. It does not operate on a dynamic population from which the worst R% are periodically deleted. The "population" in QD is effectively the archive itself, which typically only grows or improves as new, unexplored niches are filled or existing elites are surpassed. While QD is a powerful method for maintaining diversity, it does so by changing the goal of the search from finding a single optimum to illuminating a space of possibilities. The Forgetting Engine, by contrast, attempts to solve the diversity problem (via paradox retention) while still pursuing the traditional goal of finding a single, globally optimal solution.

Diversity-Promoting Selection Mechanisms

The field of evolutionary computation has developed numerous mechanisms to combat the loss of diversity. One of the most relevant is Lexicase Selection. Lexicase selection is a parent selection method that avoids aggregating multiple performance criteria (e.g., performance on different test cases) into a single scalar fitness value. Instead, for each parent selection event, it considers the test cases one by one in a random order. At each step, it filters the current pool of candidates, keeping only those that perform best on the current test case. This continues until only one candidate remains, which is then selected as a parent. This process is known to maintain population diversity by giving "specialists"—individuals that perform exceptionally well on a small, difficult subset of cases but poorly overall—a chance to be selected for reproduction.

The critical distinction between Lexicase Selection and the Forgetting Engine's mechanism lies in their role within the evolutionary loop. Lexicase is a method for **parent selection**; it determines which individuals get to *reproduce*. The Forgetting Engine's core mechanism is a method for **survivor selection**; it directly determines which individuals are *culled* from the population and do not survive to the next generation. While both ultimately promote diversity, they operate at different stages of the evolutionary cycle and through entirely different logical processes.

Conventional Selection and Pruning Methods

Standard evolutionary algorithms and search heuristics have long used concepts of selection and pruning.

- **Truncation Selection:** This is a basic selection method in genetic algorithms where the population is sorted by fitness, and only a top fraction (e.g., the top 50%) is selected to act as parents for the next generation. This is mathematically equivalent to eliminating the worst fraction of the population. However, truncation selection is a well-known and foundational technique, and its primary drawback is also well-known: it exerts very high selection pressure, leading to a rapid loss of population diversity and a high risk of premature convergence to local optima. The novelty of the Forgetting Engine relative to simple truncation selection is the addition of the **Paradox Retention** mechanism. This mechanism is a specific, non-obvious solution designed to counteract the known primary weakness of truncation, transforming a simple selection scheme into a complete, high-performance optimization engine.
- **Beam Search:** This is a heuristic search algorithm that explores a search space by expanding a limited set of the most promising nodes at each level. It maintains a "beam" of the top-K best candidate paths and discards all others. While this involves pruning, the logic is to keep the *absolute best K* candidates, not to delete a *relative worst R%* of a population of complete solutions. It is a path-finding optimization, not a population-based evolutionary algorithm. While variants of beam search have been developed to encourage diversity, they do not employ a paradox retention mechanism analogous to the Forgetting Engine's.

Artificial Immune Systems (AIS)

The term "Negative Selection Algorithm" (NSA) appears in the field of Artificial Immune Systems and bears a superficial terminological similarity to the elimination-based approach of the Forgetting Engine. However, the underlying mechanism is entirely different. NSA is an algorithm for anomaly or novelty detection, not optimization. It is inspired by the process in the biological immune system where T-cells that react to the body's own "self" proteins are eliminated. In the algorithm, a set of "detectors" is generated randomly. These detectors are then compared against a dataset of known "self" (normal) patterns. Any detector that matches a self pattern is discarded. The surviving set of detectors, which by definition only recognizes "non-self," is then used to classify new data as either normal or anomalous. The "elimination" happens to the detectors, not to a population of candidate solutions to an optimization problem. The overlap is purely semantic and does not constitute relevant prior art.

Relevant Patent Literature

A search of the patent literature did not reveal any documents that anticipate the core claims of the Forgetting Engine. The patents found in the domain of computational optimization describe different methods. For example, US Patent 8,006,220 B2 describes multi-objective optimization using surrogate models and uncertainty maximization. US Patent Application 2005/0246148 A1 describes a method for excluding non-prospective sub-regions of a domain to find Pareto-optimal points. Other patents relate to constraint satisfaction in test generation or quantum preconditioning for optimization problems. None of these documents describe a population-based algorithm that operates via an iterative loop of culling a ranked-worst percentage of candidate solutions while explicitly retaining paradoxical outliers based on

structural or diversity metrics.

Shortlist of Most Relevant Prior Art

The most pertinent prior art, which a patent examiner would most likely cite against the Forgetting Engine, are Extremal Optimization and Viability Evolution. These two approaches share the closest philosophical and, in some respects, mechanistic similarities to the invention. A direct, detailed comparison is essential to clearly delineate the inventive step.

Key Claim Element	**Forgetting Engine (Invention)**	**Extremal Optimization (Boettcher & Percus)**	**Viability Evolution (Maesani et al.)**
Forgetting-First Primitive	Operates on a **population** of candidate solutions. In each iteration, it ranks all solutions by a performance metric and **permanently eliminates the worst R%** of the entire population. This is the primary driver of convergence.	Operates on a **single** candidate solution. In each iteration, it ranks the **components** (variables) within that one solution and **modifies the state of the single worst component**. It does not eliminate entire solutions or operate on a population in its canonical form.	Operates on a **population** of candidate solutions. In each iteration, it modifies viability boundaries to **eliminate a fraction of the population that violates these hard constraints**. The elimination is based on constraint satisfaction, not ranked performance.
Paradox Retention	Includes an **explicit, feature-aware mechanism** that is applied before elimination. It identifies and **protects candidates within the worst R%** if they meet specific criteria for long-term promise (e.g., structural similarity to optima, high diversity contribution). This actively preserves exploration pathways.	**No explicit mechanism**. Exploration is an emergent property of the "avalanche" dynamics created by unconditionally accepting random changes to the worst component. It cannot deterministically protect a component based on its global context or structural promise.	**No mechanism**. An individual is either viable or non-viable. There is no described method to rescue a non-viable candidate, regardless of its novelty or other potentially promising features. All individuals within the viable set are treated as equals.
Population-Based Architecture	**Yes**. The algorithm is fundamentally population-based, maintaining a set of N parallel candidate solutions that are subject to the culling,	**No**. The canonical algorithm is a single-solution local search heuristic. "Population-Based EO" variants exist but typically use EO as a	**Yes**. The algorithm operates on a population of solutions, and its elimination mechanism is defined at the population level. The population size is

Key Claim Element	**Forgetting Engine (Invention)**	**Extremal Optimization (Boettcher & Percus)**	**Viability Evolution (Maesani et al.)**
	retention, and replenishment cycle.	mutation/local search operator on individuals, not as the primary population culling mechanism.	variable.
Overall Goal	**Optimization**. To find a single optimal or near-optimal solution to a complex problem (e.g., the lowest-energy protein fold) by efficiently navigating a deceptive search space.	**Optimization**. To find a high-quality solution to a combinatorial optimization problem by exploring the configuration space of a single candidate solution.	**Illumination / Exploration**. To find the largest possible set of diverse solutions that all satisfy a set of minimal performance/viability requirements.

Deep Dive: Extremal Optimization (Boettcher & Percus, ~2000)

Extremal Optimization is a powerful heuristic that shares the Forgetting Engine's core philosophy of focusing on negative selection. However, the comparison reveals a critical difference in the unit of selection. EO identifies and modifies the worst *part* of the *best-so-far* solution. The Forgetting Engine identifies and eliminates the worst *solutions* from an entire *population* of solutions. This is not a trivial distinction; it is a fundamental architectural choice that leads to different search dynamics. An argument that it would be obvious to scale EO's principle from a component to a population can be rebutted by the existing literature on Population-Based EO. This literature shows that the path researchers actually took was to hybridize EO with existing population-based frameworks (e.g., using EO as an operator within a Genetic Algorithm), rather than inventing the Forgetting Engine's distinct culling-based population dynamic. Furthermore, EO lacks the explicit, intelligent filtering of the paradox retention mechanism, relying instead on stochastic "avalanches" for exploration.

Deep Dive: Artificial Evolution by Viability Rather than Competition (Maesani et al., 2014)

Viability Evolution presents the closest mechanistic parallel to the Forgetting Engine's culling step. Both algorithms eliminate a fraction of the population in each evolutionary cycle. However, the differences are profound and central to the inventive concept. ViE's elimination is based on hard, binary constraints, whereas the Forgetting Engine uses a more nuanced, rank-based performance metric. This allows the Forgetting Engine to exert a continuous selection pressure towards an optimum, rather than simply ensuring solutions stay within a viable region.
Most importantly, ViE completely lacks the paradox retention mechanism. This absence stems from its different goal. ViE seeks to illuminate the space of *all viable solutions*, a task for which paradox retention is unnecessary; a solution is either viable or it is not. The Forgetting Engine seeks to find the *single best solution*, a task for which navigating deceptive landscapes is critical. Paradox retention is the non-obvious key that allows the algorithm to pair the aggressive convergence of culling with the necessary exploration to solve such problems. The combination of features in the Forgetting Engine is therefore tailored to a different, and often more difficult,

objective than that of Viability Evolution.

Gap Analysis: Articulating Novelty and Non-Obviousness

The Novelty Gap: A Unique Synthesis

The prior art contains individual elements that echo aspects of the Forgetting Engine, but no single reference or combination of references discloses the specific, integrated system claimed. The invention occupies a clear "white space" in the landscape of optimization algorithms.

- **Gap 1 (vs. Extremal Optimization):** The primary gap is the transition from a single-solution, component-level modification heuristic to a true population-based algorithm where entire solutions are the unit of selection and elimination. EO modifies the "what" (a variable's state), while the Forgetting Engine modifies the "who" (a solution's existence in the population).
- **Gap 2 (vs. Viability Evolution):** The gap consists of two key inventive steps: first, the use of a rank-based performance metric for culling instead of hard viability constraints, and second, the inclusion of the entire Paradox Retention mechanism, which is completely absent in Viability Evolution.
- **Gap 3 (vs. Truncation Selection):** The gap is the Paradox Retention mechanism itself. The prior art clearly establishes that simple truncation selection is a known technique that suffers from a well-understood flaw: the rapid loss of diversity. The Forgetting Engine's paradox retention is a specific, feature-aware solution that directly remedies this known flaw, transforming a basic selection operator into a complete, high-performance optimization engine.
- **Gap 4 (vs. Quality-Diversity & Lexicase Selection):** The gap is the fundamental algorithmic architecture and objective. The Forgetting Engine employs a "cull-and-replenish" dynamic to find a single optimum. This is distinct from QD's "archive-and-fill" dynamic for illumination and Lexicase's role as a filter for parent selection.

The Non-Obviousness Argument: A Synergistic System

The argument for non-obviousness rests on the synergistic effect created by combining two opposing forces: aggressive culling and paradox retention. It is well-known in the art that aggressive selection pressure, such as that provided by culling the worst 30% of the population, leads to rapid convergence (exploitation). It is equally well-known that this same pressure is highly destructive to population diversity and often leads to premature convergence on suboptimal solutions, especially in rugged or deceptive fitness landscapes. The challenge of balancing exploitation and exploration is one of the oldest and most fundamental problems in evolutionary computation.

The Forgetting Engine presents a non-obvious solution to this problem. Instead of weakening the selection pressure to preserve diversity, it maintains an extremely high selection pressure (the culling step) and introduces a highly specific, intelligent counter-mechanism (paradox retention) to preserve only the most valuable diversity. The paradox retention filter does not simply keep random or diverse individuals; it specifically targets individuals that are performing

poorly *now* but have structural or positional features that suggest they are part of a difficult but ultimately more promising path toward the global optimum.
This combination creates a system that achieves a result that would not be expected by one of ordinary skill in the art. It produces an algorithm that is both significantly faster than a traditional exploratory method like Monte Carlo (a result of the aggressive culling) and has a significantly higher success rate in finding the global optimum (a result of the paradox retention), as demonstrated by the experimental data. This ability to successfully combine two seemingly contradictory pressures to achieve superior results in both speed and accuracy is a hallmark of a non-obvious invention. The solution is not a mere aggregation of known elements but an integrated system where the components work synergistically to overcome each other's known limitations.

Strategic Recommendations and Path Forward

Claim Drafting Strategy

To ensure the broadest defensible patent protection, the claims must be drafted to capture the full, unique operational loop of the Forgetting Engine.

- **Independent Claims:** The independent claims (as exemplified by Claim 1 and Claim 2 in the provisional application) are well-formed and should be the foundation of the non-provisional application. They must recite the complete, synergistic system:
    1. The context of a **population** of candidate solutions.
    2. The iterative step of evaluating and identifying a **subset of worst-performing** candidates based on a ranked objective function.
    3. The step of **permanently eliminating** said subset from the population.
    4. The crucial step of identifying and **retaining** specific candidates from within that worst-performing subset based on their possession of **contradictory or paradoxical properties** (e.g., promising structural features despite poor current performance).
    5. The step of replenishing the population.
- **Functional Language:** While the specification should fully describe the "Forgetting Engine" and its philosophy, the claims should use clear, functional language, such as "a strategic elimination module" and "a paradox retention module," to define the components of the system. This avoids limiting the claims to a specific brand name and focuses on the novel function.
- **Dependent Claims:** A robust set of dependent claims should be drafted to cover specific embodiments and features. These should include the preferred percentage for elimination (e.g., approximately 20-40%) , the typical population sizes (e.g., 20-100 candidates) , the specific criteria for paradox retention (SCS, EPI, DCM) , and the validated application to protein folding using HP energy models.

Navigating Key Prior Art in Prosecution

During patent prosecution, it is highly likely that an examiner will cite Extremal Optimization and/or Viability Evolution. The following arguments should be prepared:

- **Against an Extremal Optimization (EO) Rejection:** The primary rebuttal is the fundamental architectural difference. Argue that EO teaches the modification of

components within a single solution, whereas the invention claims the elimination of *entire solutions from a population*. Use the Population-Based EO literature as evidence that the "obvious" way to extend EO to a population, as practiced by others in the field, was through hybridization, not through the invention's novel culling-and-retention dynamic.

- **Against a Viability Evolution (ViE) Rejection:** The rebuttal should focus on two points. First, distinguish the elimination criteria: the invention uses **ranked performance**, while ViE uses **hard constraint violation**. Second, and more importantly, emphasize the complete absence of the **Paradox Retention mechanism** in ViE. Argue that adding this mechanism would not have been obvious because it serves a fundamentally different goal (optimization vs. illumination). The purpose of paradox retention is to solve deceptive optimization problems, a challenge not addressed or contemplated by the ViE disclosure, which is focused on finding a diverse set of merely viable solutions.

Suggestions for Further Work to Strengthen Application

To further bolster the patent application against potential challenges and strengthen the case for non-obviousness, the following experimental work is recommended:

1. **Direct Comparative Experiments:** Conduct new experiments directly comparing the Forgetting Engine against both a Population-Based EO variant and the Viability Evolution algorithm on the exact same 2D HP lattice protein folding benchmark used in the original study. Demonstrating statistically significant superior performance (in speed, success rate, and final energy) against the closest prior art would provide powerful, direct evidence of an unexpected result and non-obviousness.
2. **Demonstrate Generality:** Apply the Forgetting Engine to at least one other well-known NP-hard problem from a different domain, such as the Traveling Salesman Problem (TSP) or a scheduling optimization problem. Successfully demonstrating the algorithm's effectiveness beyond protein folding will strengthen the claims for broader applicability and underscore its status as a general-purpose optimization method.
3. **Characterize Retained Candidates:** Perform an analysis of a series of optimization runs and track the candidates that are saved by the paradox retention mechanism. Demonstrate statistically that these "paradoxical" individuals are significantly more likely to be ancestors of the final optimal solution compared to other individuals with similar poor fitness that were not retained. This would provide strong empirical evidence for the efficacy and non-obviousness of the retention mechanism itself.

Works cited

1. Stefan Boettcher - Emory University, https://faculty.college.emory.edu/sites/boettcher/Research/eo.htm 2. Artificial Evolution by Viability Rather than Competition | PLOS One - Research journals, https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0086831 3. A New Algorithm for 2D Hydrophobic-Polar Model: An Algorithm Based on Hydrophobic Core in Square Lattice - Science Alert, https://scialert.net/fulltext/?doi=pjbs.2008.1815.1819 4. Generalized Extremal Optimization for Solving Complex Optimal Design Problems, https://www.researchgate.net/publication/225169785_Generalized_Extremal_Optimization_for_Solving_Complex_Optimal_Design_Problems 5. Extremal optimization - Wikipedia, https://en.wikipedia.org/wiki/Extremal_optimization 6. Optimization with Extremal Dynamics | Phys. Rev. Lett. - Physical Review Link Manager,

https://link.aps.org/doi/10.1103/PhysRevLett.86.5211 7. Optimization with Extremal Dynamics - CGU Scholar, https://scholar.cgu.edu/allon-percus/wp-content/uploads/sites/11/2013/08/complexity.pdf 8. Studies on Extremal Optimization and Its Applications in Solving Real World Optimization Problems - ViGIR-lab, http://vigir.missouri.edu/~gdesouza/Research/Conference_CDs/IEEE_SSCI_2007/Foundations%20of%20CI%20-%20FOCI%202007/data/papers/FOCI/S001P024.pdf 9. A novel particle swarm optimizer hybridized with extremal optimization, https://optimization-online.org/wp-content/uploads/2008/05/1993.pdf 10. Continuous extremal optimization for Lennard-Jones clusters, http://staff.ustc.edu.cn/~clj/pdf/PRE1.pdf 11. Techniques for Monte Carlo Optimizing, Monte Carlo Methods and Applications, 4(3), 181-230, 1998. - ResearchGate, https://www.researchgate.net/publication/345150544_Techniques_for_Monte_Carlo_Optimizing_Monte_Carlo_Methods_and_Applications_43_181-230_1998 12. EVOLUTIONARY DYNAMICS OF EXTREMAL OPTIMIZATION - SciTePress, https://www.scitepress.org/papers/2009/23141/23141.pdf 13. (PDF) A Population-Based Hybrid Extremal Optimization Algorithm, https://www.researchgate.net/publication/220776251_A_Population-Based_Hybrid_Extremal_Optimization_Algorithm 14. Multiobjective optimization using population-based extremal optimization | Request PDF, https://www.researchgate.net/publication/220372655_Multiobjective_optimization_using_population-based_extremal_optimization 15. [PDF] Extremal Optimization: Heuristics Via Co-Evolutionary Avalanches | Semantic Scholar, https://www.semanticscholar.org/paper/Extremal-Optimization%3A-Heuristics-Via-Avalanches-Boettcher/7a2d0834a604f3a2a6da07618150f5c61034b98c 16. Artificial Evolution by Viability Rather than Competition - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC3906060/ 17. Quality Diversity: A New Frontier for Evolutionary ... - Frontiers, https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2016.00040/full 18. Procedural Content Generation through Quality Diversity - Antonios Liapis, https://antoniosliapis.com/articles/pcgqd.php 19. MAP-Elites discretizes the feature space according to some... - ResearchGate, https://www.researchgate.net/figure/MAP-Elites-discretizes-the-feature-space-according-to-some-pre-specified-number-of_fig1_318894662 20. DALex: Lexicase-like Selection via Diverse AggregationSupported by Amherst College and members of the PUSH lab. - arXiv, https://arxiv.org/html/2401.12424v2 21. Lexicase Selection Beyond Genetic Programming - Faculty, https://faculty.hampshire.edu/lspector/pubs/lexicase-beyond-gp-preprint.pdf 22. A probabilistic and multi-objective analysis of lexicase selection and ϵ-lexicase selection - PMC - PubMed Central, https://pmc.ncbi.nlm.nih.gov/articles/PMC9453780/ 23. Artificial selection methods from evolutionary computing show promise for directed evolution of microbes - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC9444240/ 24. Artificial selection methods from evolutionary computing show promise for directed evolution of microbes | eLife, https://elifesciences.org/articles/79665 25. Selection (evolutionary algorithm) - Wikipedia, https://en.wikipedia.org/wiki/Selection_(evolutionary_algorithm) 26. (PDF) Selection Methods for Genetic Algorithms - ResearchGate, https://www.researchgate.net/publication/259461147_Selection_Methods_for_Genetic_Algorithms 27. Evolutionary Algorithms 3 Selection - GEATbx, http://www.geatbx.com/docu/algindex-02.html 28. Information-Geometric Optimization with Natural Selection - PMC - PubMed Central, https://pmc.ncbi.nlm.nih.gov/articles/PMC7597266/

29. The Role and Application of Beam Search in Modern Machine Learning - Uncodemy, https://uncodemy.com/blog/beam-search 30. Hierarchical Beam Search for Solving Most Relevant Explanation in Bayesian Networks, https://cdn.aaai.org/ocs/10370/10370-46369-1-PB.pdf 31. Modern Beam Search Techniques - Emergent Mind, https://www.emergentmind.com/topics/beam-search-techniques 32. Diverse Beam Search: Decoding Diverse Solutions from Neural Sequence Models - Semantic Scholar, https://www.semanticscholar.org/paper/Diverse-Beam-Search%3A-Decoding-Diverse-Solutions-Vijayakumar-Cogswell/e4dd95c4341ec7d14317a3d97022773a0822906c 33. Negative Selection Algorithm, https://algorithmafternoon.com/immune/negative_selection_algorithm/ 34. A Cuckoo Search Detector Generation-based Negative Selection Algorithm - Tech Science Press, https://www.techscience.com/csse/v38n2/42337/html 35. Artificial Immune System (AIS): A Guide With Python Examples - DataCamp, https://www.datacamp.com/tutorial/artificial-immune-system 36. A Self-Adaptive Evolutionary Negative Selection Approach for Anomaly Detection - NSUWorks, https://nsuworks.nova.edu/cgi/viewcontent.cgi?article=1688&context=gscis_etd 37. US8006220B2 - Model-building optimization - Google Patents, https://patents.google.com/patent/US8006220B2/en 38. US20050246148A1 - Exclusion of regions method for multi-objective optimization - Google, https://www.google.com.na/patents/US20050246148 39. US20130007680A1 - Coverage Based Pairwise Test Set Generation for Verification of Electronic Designs - Google Patents, https://patents.google.com/patent/US20130007680A1/en 40. Optimization via quantum preconditioning | Phys. Rev. Applied, https://link.aps.org/doi/10.1103/9prw-684p